

1st Attempt (incorrect answer)

Description: -

This was our 1st attempt before the modification of the H.W. question, at the time we thought when the H.W. said "choose action a1 in all states" and "choose action a2 in all states" would be comparison between these 2 policies. However, it turns that this was incorrect.

Foot Note: -

We decided to leave the code here, since we already answered it before the modification of the H.W.

Important Note: -

The correct answer is the 2nd attempt below.

```
import numpy as np

# Created a transition probability 2d array for policy/action 1
poli_a1 = np.array([
    [0.0, 0.3, 0.7, 0.0, 0.0, 0.0],
    [0.0, 0.3, 0.7, 0.0, 0.0, 0.0],
    [0.4, 0.0, 0.0, 0.6, 0.0, 0.0],
    [0.0, 0.0, 0.0, 0.0, 0.6, 0.4],
    [0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
    [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
])

# Created a transition probability 2d array for policy/action 2
poli_a2 = np.array([
    [0.3, 0.0, 0.0, 0.7, 0.0, 0.0],
    [0.3, 0.0, 0.0, 0.7, 0.0, 0.0],
    [0.0, 0.4, 0.6, 0.0, 0.0, 0.0],
    [0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
    [0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
    [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
])

# The rewards assigned to each state and the discount factor
rewards = np.array([1, 1, 1, 1, 0, 0])
gamma = 0.7

epsilon = .01
iterations = 5
```

```

def compute_values(policy):
    # Initializing all of the states to 0
    value = np.zeros(6)

    for i in range(iterations):
        previous_value = np.copy(value)
        for state in range(6):
            if state < 4:
                value[state] = rewards[state] + gamma * np.sum(policy[state] *
previous_value)
            else:
                value[state] = rewards[state]

        # This part of the code serves as a convergence criterion.
Basically, by exiting the loop, it prevents any unnecessary
computation
        if np.max(np.abs(value - previous_value)) < epsilon:
            break

    return value

value_a1 = compute_values(poli_a1)
print("Values for a1 V(s):", value_a1)
value_a2 = compute_values(poli_a2)
print("Values for a1 V(s):", value_a2)

def compare_policies(value_a1, value_a2):
    optimal_policy = None

    # We're counting for the amount of values that are greater in each
policy
    count_a1 = np.sum(value_a1 > value_a2) # In this case, it would be
s1 and s2
    count_a2 = np.sum(value_a2 > value_a1) # In this case, it would be
s3

    if count_a1 > count_a2:
        optimal_policy = "π(a1) is a more optimal policy."
    elif count_a2 > count_a1:
        optimal_policy = "π(a2) is a more optimal policy."
    else:
        optimal_policy = "Both policies are equivalent"

    return optimal_policy

rst = compare_policies(value_a1, value_a2)
print("\n", rst)

```

```
Values for a1 V(s): [2.4979454 2.4979454 2.0889032 1. 0.
0. ]
Values for a1 V(s): [1.8843527 1.8843527 2.5493268 1. 0.
0. ]
```

$\pi(a1)$ is a more optimal policy.

2nd Attempt (correct answer)

Description: -

Utilized the *professor's code* and *Chatgpt* to best optimize the process of finding the best optimal value in each state.

Foot Note: -

This code was done after the modification of the H.W.'s question. Thus, we found out that we made a mistake at the previous attempt, meaning we had to start over again. This is our final answer as it reveals the optimal policy.

```
class Action:
    """
    Represents an action in the Markov Decision Process (MDP).

    Attributes:
        name (str): The name of the action.
        n_nodes (list): List of next state names for each possible
        outcome.
        prob (list): List of transition probabilities corresponding to
        n_nodes.
    """

    def __init__(self, name, n_nodes, prob):
        """
        Initializes an Action object.

        Args:
            name (str): The name of the action.
            n_nodes (list): List of next state names.
            prob (list): List of transition probabilities.
        """
        self.name = name
        self.n_nodes = n_nodes
        self.prob = prob

    def get_rst(self):
        """
```

```

    Returns a list representing the action's transitions.

    Returns:
        list: List containing tuples of (next_state, probability).
    """
    rst = []
    for n_nodes, prob in zip(self.n_nodes, self.prob):
        rst.append((n_nodes, prob))
    rst.insert(0, self.name)
    return rst

class State:
    """
    Represents a state in the Markov Decision Process (MDP).

    Attributes:
        name (str): The name of the state.
        reward (float): The reward associated with being in this
state.
        action (list): List of Action objects available from this
state.
        value (float): The current value of the state under a policy.
    """

    def __init__(self, name, reward, action, value=0):
        """
        Initializes a State object.

        Args:
            name (str): The name of the state.
            reward (float): The reward associated with this state.
            action (list): List of Action objects available from this
state.
            value (float, optional): The initial value of the state.
Defaults to 0.
        """
        self.name = name
        self.reward = reward
        self.action = action
        self.value = value

    def get_action(self):
        """
        Returns a list of possible actions from this state.

        Returns:
            list: List of tuples representing actions and their
transitions.
        """
        if self.action is None:

```

```

        return []

    moves = []
    for a in self.action:
        moves.append(a.get_rst())
    return moves

class Problem:
    """
    Represents a Markov Decision Process (MDP) problem.

    Attributes:
        states (list): List of State objects representing the states
in the MDP.
    """

    def __init__(self, states):
        """
        Initializes a Problem object.

        Args:
            states (list): List of State objects representing the
states in the MDP.
        """
        self.states = states

def policy_iteration(problem, niter=5, gamma=1, epsilon=0.01):
    """
    Perform policy iteration to find the optimal policy for the given
MDP problem.

    Args:
        problem (Problem): The Markov Decision Process (MDP) problem
to solve.
        niter (int): Maximum number of iterations for policy
iteration. Defaults to 5.
        gamma (float): Discount factor for future rewards. Defaults to
1.
        epsilon (float): Threshold for convergence. Defaults to 0.01.

    Returns:
        dict: A dictionary mapping each state name to a tuple
containing the optimal action
and the computed value.
    """
    actions = {state.name: None for state in problem.states}

    for state in problem.states:
        state.value = 0

```

```

    for i in range(niter):
        previous_values = {state.name: state.value for state in
problem.states}

        for state in problem.states:
            if state.action is None:
                state.value = state.reward
            else:
                a_values = []
                for action in state.get_action():
                    if len(action) > 1: # Ensure there are
transitions defined
                        optimal_v = sum(prob * previous_values[node]
for node, prob in action[1:])
                        a_values.append((action[0], optimal_v))

                if not a_values:
                    raise ValueError(f"No valid actions defined for
state {state.name}")

                optimal_a, optimal_v = max(a_values, key=lambda x:
x[1])

                state.value = state.reward + gamma * optimal_v
                actions[state.name] = optimal_a

            delta = max(abs(state.value - previous_values[state.name]) for
state in problem.states)
            if delta <= epsilon * (1 - gamma) / gamma:
                print("Values converged after", i + 1, "iterations")
                break
            else:
                print("Not converged after", niter, "iterations")

        return {state.name: (actions[state.name], state.value) for state
in problem.states}

# Define your states and actions
s1 = State(
    name="S1",
    reward=1,
    action=[
        Action(
            name="a1",
            n_nodes=["S2", "S3"],
            prob=[0.3, 0.7]
        ),
        Action(
            name="a2",
            n_nodes=["S1", "S4"],
            prob=[0.3, 0.7]

```

```

    )
]
)

s2 = State(
    name="S2",
    reward=1,
    action=[
        Action(
            name="a1",
            n_nodes=["S2", "S3"],
            prob=[0.3, 0.7]
        ),
        Action(
            name="a2",
            n_nodes=["S1", "S4"],
            prob=[0.3, 0.7]
        )
    ]
)

s3 = State(
    name="S3",
    reward=1,
    action=[
        Action(
            name="a1",
            n_nodes=["S1", "S4"],
            prob=[0.4, 0.6]
        ),
        Action(
            name="a2",
            n_nodes=["S2", "S3"],
            prob=[0.4, 0.6]
        )
    ]
)

s4 = State(
    name="S4",
    reward=1,
    action=[
        Action(
            name="a1",
            n_nodes=["S5", "S6"],
            prob=[0.6, 0.4]
        )
    ]
)

```

```

s5 = State(
    name="S5",
    reward=0,
    action=None
)

s6 = State(
    name="S6",
    reward=0,
    action=None
)

# Create the Problem instance
problem = Problem([s1, s2, s3, s4, s5, s6])

# Run policy iteration
rst = policy_iteration(problem, niter=17, gamma=0.7, epsilon=0.01)

# Print results
for state, value in rst.items():
    print(f"State: {state}")
    print(f"Action taken: {value[0]}")
    print(f"Value: {value[1]}")
    print(f"-----")

```

Values converged after 17 iterations

State: S1

Action taken: a1

Value: 3.3255789828670923

State: S2

Action taken: a1

Value: 3.3255789828670923

State: S3

Action taken: a2

Value: 3.3255789828670923

State: S4

Action taken: a1

Value: 1.0

State: S5

Action taken: None

Value: 0

State: S6

Action taken: None

Value: 0

Iteration 0

```
rst = policy_iteration(problem, niter=0, gamma=0.7, epsilon=0.01)

# Print results
for state, value in rst.items():
    print(f"State: {state}")
    print(f"Action taken: {value[0]}")
    print(f"Value: {value[1]}")
    print(f"-----")
```

Not converged after 0 iterations

State: S1

Action taken: None

Value: 0

State: S2

Action taken: None

Value: 0

State: S3

Action taken: None

Value: 0

State: S4

Action taken: None

Value: 0

State: S5

Action taken: None

Value: 0

State: S6

Action taken: None

Value: 0

Iteration 1

```
rst = policy_iteration(problem, niter=1, gamma=0.7, epsilon=0.01)

# Print results
for state, value in rst.items():
    print(f"State: {state}")
    print(f"Action taken: {value[0]}")
    print(f"Value: {value[1]}")
    print(f"-----")
```

Not converged after 1 iterations

State: S1

Action taken: a1

Value: 1.0

State: S2

Action taken: a1

Value: 1.0

State: S3

Action taken: a1

Value: 1.0

State: S4

Action taken: a1

Value: 1.0

State: S5

Action taken: None

Value: 0

State: S6

Action taken: None

Value: 0

Iteration 2

```
rst = policy_iteration(problem, niter=2, gamma=0.7, epsilon=0.01)
```

```
# Print results
```

```
for state, value in rst.items():
```

```
    print(f"State: {state}")
```

```
    print(f"Action taken: {value[0]}")
```

```
    print(f"Value: {value[1]}")
```

```
    print(f"-----")
```

Not converged after 2 iterations

State: S1

Action taken: a1

Value: 1.7

State: S2

Action taken: a1

Value: 1.7

State: S3

Action taken: a1

Value: 1.7

State: S4

Action taken: a1

Value: 1.0

State: S5

Action taken: None

Value: 0

State: S6

Action taken: None

Value: 0

Iteration 3

```
rst = policy_iteration(problem, niter=3, gamma=0.7, epsilon=0.01)
```

```
# Print results
```

```
for state, value in rst.items():
```

```
    print(f"State: {state}")
```

```
    print(f"Action taken: {value[0]}")
```

```
    print(f"Value: {value[1]}")
```

```
    print(f"-----")
```

Not converged after 3 iterations

State: S1

Action taken: a1

Value: 2.19

State: S2

Action taken: a1

Value: 2.19

State: S3

Action taken: a2

Value: 2.19

State: S4

Action taken: a1

Value: 1.0

State: S5

Action taken: None

Value: 0

State: S6

```
Action taken: None
Value: 0
-----
```

Iteration 4

```
rst = policy_iteration(problem, niter=4, gamma=0.7, epsilon=0.01)

# Print results
for state, value in rst.items():
    print(f"State: {state}")
    print(f"Action taken: {value[0]}")
    print(f"Value: {value[1]}")
    print(f"-----")
```

Not converged after 4 iterations

```
State: S1
Action taken: a1
Value: 2.533
-----
```

```
State: S2
Action taken: a1
Value: 2.533
-----
```

```
State: S3
Action taken: a2
Value: 2.533
-----
```

```
State: S4
Action taken: a1
Value: 1.0
-----
```

```
State: S5
Action taken: None
Value: 0
-----
```

```
State: S6
Action taken: None
Value: 0
-----
```

Iteration 5 (Final Answer)

```
rst = policy_iteration(problem, niter=5, gamma=0.7, epsilon=0.01)
```

```
# Print results
```

```
for state, value in rst.items():  
    print(f"State: {state}")  
    print(f"Action taken: {value[0]}")  
    print(f"Value: {value[1]}")  
    print(f"-----")
```

Not converged after 5 iterations

State: S1

Action taken: a1

Value: 2.7731

State: S2

Action taken: a1

Value: 2.7731

State: S3

Action taken: a2

Value: 2.7731

State: S4

Action taken: a1

Value: 1.0

State: S5

Action taken: None

Value: 0

State: S6

Action taken: None

Value: 0
